

Container mit Schwerpunkt Kubernetes

BSI-Rahmen für die Container- Lifecycle-Architektur



Agenda

1. Harbor als Container Artifactory

- Harbor als Beispiel für eine Container-Artifactory
- Sicherheit von Container-Images überprüfen: Vulnarability-Scanner
- Image Signing
- Policy Enforcement

BSI-Rahmen für den Container-Lifecycle

Einordnung: BSI, IT-Grundschutz und Containerisierung

- **Das Bundesamt für Sicherheit in der Informationstechnik (BSI)**

Das **Bundesamt für Sicherheit in der Informationstechnik (BSI)** ist die zentrale Cyber-Sicherheitsbehörde des Bundes.

Es definiert verbindliche und empfohlene Sicherheitsstandards für Behörden und Unternehmen in Deutschland, insbesondere für:

- den sicheren Betrieb von IT-Systemen
- die Absicherung moderner IT-Architekturen
- die Einhaltung von Compliance- und Schutzanforderungen

Ein zentrales Werkzeug hierfür ist der **IT-Grundschutz**.

BSI-Rahmen für den Container-Lifecycle

■ Das IT-Grundschutz-Kompendium

Das **IT-Grundschutz-Kompendium** beschreibt **konkrete, umsetzbare Sicherheitsanforderungen** für IT-Systeme, Anwendungen und Plattformen.

Charakteristisch für den Grundschutz ist:

- praxisnahe Maßnahmen
- technikneutral formuliert
- lebenszyklusorientierter Ansatz

Wichtig:

Der IT-Grundschutz schreibt *nicht vor, welche Tools* zu verwenden sind, sondern *welche Sicherheitsziele erreicht werden müssen*.

Container-Plattformen wie Kubernetes werden dabei **nicht als Sonderfall**, sondern als reguläre IT-Systeme betrachtet – mit besonderen Risiken.

https://www.bsi.bund.de/DE/Themen/Unternehmen-und-Organisationen/Standards-und-Zertifizierung/IT-Grundschutz/IT-Grundschutz-Kompendium/it-grundschutz-kompendium_node.html

BSI-Rahmen für den Container-Lifecycle

■ Der Container-Lifecycle aus BSI-Sicht

Das BSI betrachtet Container nicht nur zur Laufzeit, sondern über ihren **gesamten Lebenszyklus** hinweg.

Ein typischer Container-Lifecycle umfasst:

- | | |
|-------------------------------|---------------------------------------|
| 1. Entwicklung der Anwendung | 4. Deployment in die Laufzeitumgebung |
| 2. Build des Container Images | 5. Betrieb, Updates und Wartung |
| 3. Ablage in einer Registry | 6. Außerbetriebnahme |

Zentrale BSI-Aussage:

Sicherheitsmaßnahmen müssen **frühzeitig und kontinuierlich** entlang dieses Lebenszyklus greifen.

Gerade Container-Registries und automatisierte Deployments sind dabei **kritische Kontrollpunkte**.

Werden hier keine Sicherheitsmechanismen etabliert, können sich Fehler oder Angriffe sehr schnell auf ganze Cluster ausbreiten.

Umsetzung gemäß SYS.1.6 – Systemmanagement

■ SYS.1.6 im Kontext von Container-Umgebungen

Der Baustein SYS.1.6 Systemmanagement beschreibt grundlegende Anforderungen an den sicheren und geordneten Betrieb von IT-Systemen. Ziel ist es, Systeme so zu betreiben, dass sie jederzeit kontrollierbar, nachvollziehbar und beherrschbar bleiben.

Übertragen auf Container- und Kubernetes-Plattformen bedeutet das:

Auch hochautomatisierte und dynamische Umgebungen müssen klaren Regeln folgen. Der Betrieb darf nicht davon abhängen, dass einzelne Personen „wissen, wie es geht“.

Im Kern fordert SYS.1.6:

- klare Verantwortlichkeiten
- zentrale Steuerung statt individueller Freiheiten
- standardisierte Prozesse und vollständige Nachvollziehbarkeit
- technische Durchsetzung statt rein organisatorischer Vorgaben

https://www.bsi.bund.de/SharedDocs/Downloads/DE/BSI/Grundschatz/IT-GS-Kompendium_Einzel_PDFs_2023/07_SYS_IT_Systeme/SYS_1_6_Containerisierung_Edition_2023.html

Umsetzung gemäß SYS.1.6 – Systemmanagement

Zentrale Verwaltung und Kontrolle

SYS.1.6 fordert eine zentrale Verwaltung aller relevanten Systemkomponenten. In Container-Umgebungen betrifft dies vor allem Container-Images, da sie die eigentlichen Anwendungseinheiten darstellen.

Ohne zentrale Kontrolle besteht die Gefahr, dass Images aus unterschiedlichen Quellen stammen, ungetestet eingesetzt werden oder unbemerkt verändert werden. Eine zentrale Registry mit klaren Zugriffsrechten schafft hier Transparenz und Kontrolle.

In Container-Umgebungen betrifft das insbesondere:

- wo Container-Images herkommen
- welche Images verwendet werden dürfen
- wer Images bereitstellen oder verändern darf

Eine zentrale Container-Registry (z. B. Harbor) wird damit zum **Systemmanagement-Werkzeug**, nicht nur zum Speicher.

Umsetzung gemäß SYS.1.6 – Systemmanagement

Nachvollziehbarkeit und Änderungsmanagement

Ein zentrales Ziel von SYS.1.6 ist die **Nachvollziehbarkeit von Änderungen**.

Durch die Nutzung von Digests, Signaturen und Audit-Logs kann jederzeit nachvollzogen werden, welches Image tatsächlich im Einsatz ist und wer es bereitgestellt hat.

In Container-Umgebungen bedeutet das:

- Images müssen eindeutig identifizierbar sein (Digest statt nur Tag)
- der Ersteller eines Images muss erkennbar sein
- Änderungen am Image dürfen nicht unbemerkt bleiben

Image Signing und Audit-Logs sind hier zentrale technische Umsetzungen.

Umsetzung gemäß SYS.1.6 – Systemmanagement

Trennung von Rollen und Verantwortlichkeiten

SYS.1.6 fordert eine **klare Trennung von Aufgaben**:

- Entwicklung
- Betrieb
- Freigabe

In Container-Plattformen wird dies umgesetzt durch:

- rollenbasierte Zugriffe in der Registry
- getrennte CI-Pipelines
- restriktive Deployment-Policies

Wichtig:

**Ein Entwickler darf ein Image bauen –
aber nicht zwangsläufig in Produktion deployen.**

Umsetzung gemäß SYS.1.6 – Systemmanagement

Automatisierung statt manueller Kontrolle

Manuelle Prüfungen gelten im Grundsatz als **fehleranfällig**.
SYS.1.6 fordert daher weitgehend automatisierte Prozesse.

In der Praxis bedeutet das:

- automatisches Vulnerability Scanning
- automatisches Blockieren unsicherer Images
- Policy Enforcement im Kubernetes Admission-Prozess

Umsetzung gemäß APP.4.4 – Containerisierung von Anwendungen

Zielsetzung von APP.4.4

Der Baustein **APP.4.4 Containerisierung von Anwendungen** konkretisiert Sicherheitsanforderungen für **containerisierte Applikationen**.

Der Fokus liegt auf:

- Integrität der Anwendung
- Schutz vor Manipulation
- kontrollierter Nutzung von Container-Images

APP.4.4 adressiert damit **direkt Kubernetes-Umgebungen** und fordert daher, dass Container-Images als sicherheitskritische Artefakte behandelt werden.

Ihre Herkunft, Integrität und Aktualität müssen jederzeit überprüfbar sein.

Im Fokus steht nicht der einzelne Container, sondern das Zusammenspiel aus Image, Plattform und Betriebsprozessen.

https://www.bsi.bund.de/SharedDocs/Downloads/DE/BSI/Grundschutz/IT-GS-Kompendium_Einzel_PDFs_2023/06_APP_Anwendungen/APP_4_4_Kubernetes_Edition_2023.html

Umsetzung gemäß APP.4.4 – Containerisierung von Anwendungen

Zentrale Sicherheitsanforderungen aus APP.4.4

▪ Vertrauenswürdige Herkunft von Images

Ein zentrales Risiko in Container-Umgebungen ist die unkontrollierte Verwendung fremder oder manipulierte Images. APP.4.4 fordert daher explizit, dass nur vertrauenswürdige Container-Images eingesetzt werden dürfen.

Das bedeutet:

- definierte Quellen
- keine unkontrollierten Fremd-Images
- klare Freigabeprozesse

Technische Umsetzung:

- eigene Registry
- eingeschränkte Registry-Nutzung
- Policy Enforcement

Umsetzung gemäß APP.4.4 – Containerisierung von Anwendungen

▪ Schutz vor Image-Manipulation

Ein zentrales Risiko in Container-Umgebungen ist die **unbemerkte Manipulation von Images**.

APP.4.4 fordert daher Maßnahmen zur Sicherstellung der Integrität:

- Nutzung unveränderlicher Digests
- kryptografische Signaturen
- Überprüfung vor dem Deployment

➔ Image Signing wird damit zu einer **BSI-konformen Maßnahme**.

Umsetzung gemäß APP.4.4 – Containerisierung von Anwendungen

▪ Umgang mit bekannten Schwachstellen

Bekannte Schwachstellen stellen ein erhebliches Risiko für containerisierte Anwendungen dar, da sie sich schnell vervielfältigen können. APP.4.4 fordert deshalb einen systematischen Umgang mit Schwachstellen, der automatisierte Prüfungen einschließt.

APP.4.4 verlangt einen strukturierten Umgang mit Sicherheitslücken:

- Erkennen
- Bewerten
- Behandeln

Container-Umsetzung durch:

- integrierte Vulnerability Scanner
- Schwellwerte (z. B. CRITICAL)
- automatische Blockierung unsicherer Images

Umsetzung gemäß APP.4.4 – Containerisierung von Anwendungen

▪ Trennung von Umgebungen

Darüber hinaus verlangt der Baustein eine klare Trennung von **Entwicklungs-, Test- und Produktionsumgebungen**.

Diese Trennung ist nicht nur organisatorisch, sondern auch technisch umzusetzen – etwa durch getrennte Registry-Projekte und unterschiedliche Sicherheits-Policies.

In Container-Plattformen:

- separate Registry-Projekte
- unterschiedliche Policies
- restriktivere Regeln in Produktion

Zusammenfassung

**Der IT-Grundschutz verlangt keine speziellen Tools –
aber er verlangt Kontrolle, Nachvollziehbarkeit und Durchsetzung.**

Container-Registries, Image Signing und Kubernetes-Policies sind:

- keine „nice-to-have“-Features
- sondern **konkrete technische Umsetzungen von BSI-Anforderungen**

Die im IT-Grundschutz geforderten Maßnahmen lassen sich in Kubernetes-Umgebungen konkret und technisch umsetzen – vorausgesetzt, Sicherheitsmechanismen werden konsequent in den Container-Lifecycle integriert.

ORDIX AG
Aktiengesellschaft für Softwareentwicklung,
Schulung, Beratung und Systemintegration

Zentrale Paderborn
Karl-Schurz-Straße 19a
33100 Paderborn
Tel.: 05251 1063-0
Fax: 0180 1 67349 0

Seminarzentrum Wiesbaden
Kreuzberger Ring 13
65205 Wiesbaden
Tel.: 0611 77840-00

seminare@ordix.de
<https://seminare.ordix.de/>

Vielen Dank für Ihre Aufmerksamkeit !